第3章 MCS-51单片机指令系统

3.1 指令系统概述

3.2 寻址方式

3.3 指令系统

3.4 伪指令

3.1 指令系统概述

一、指令格式

[标号：] 操作码 [操作数1], [操作数2], [操作数3]; 注释

例： LOOP： MOV A，#40H ；立即数赋给A

- **标号：**
  - 用户定义的符号，是指令的符号地址，标号值代表这条指令在程序存储器中的存放地址
  - 标号以字符开头，后跟1~8个字符或数字
- **操作码：**
  - 指明指令功能，在汇编语言中是功能英文单词的缩写

[标号：] 操作码 [操作数1], [操作数2], [操作数3]; 注释

- **操作数：**
  - 指令操作对象
  - 根据指令不同，有1，2，4，或0个操作数不同
  - 2个操作数中，第一个为目标操作数，第二个为源操作数
- **注释：**
  - 以分号“；”开始，对指令或程序段功能的说明
- **整个语句必须在一行之内写完**

二、操作数类型

◆ 立即数 （数据前面标#）

- 可以是二进制数，十六进制数，十进制数，表达式（有确定值）
- 作为指令代码的一部分出现在指令中，通常为源操作数

◆ 寄存器操作数

- 操作数存放在寄存器中
- 指令中直接给出寄存器名称，可以是源操作数或目的操作数

◆ 存储器操作数

- 操作数存放在存储器
- 指令中给出存储器地址

三、指令描述约定

- ✓ R_n：选定的工作寄存器区中的8个寄存器R_7~R_0（n=0~7）
- ✓ Direct：8位片内数据存储器单元地址
- ✓ @R_i：通过寄存器**R_0**或**R_1**间接寻址的8位片内数据存储器单元
- ✓ #data：指令中的8位立即数
- ✓ #data16：指令中的16位立即数
- ✓ Addr16：16位目标地址（64KB）　**LCALL　LJMP**
- ✓ Addr11：11位目标地址（2KB）　**ACALL　AJMP**

三、指令描述约定(续)

- ✓ rel：带符号的8位偏移量字节（-128B~+127B） **SJMP 条件转移指令**
- ✓ bit：内部数据存储器或专用寄存器里的直接寻址位
- ✓ /bit ：表示对该位取反操作
- ✓ （X）：X中的内容
- ✓ （（X））：由X所指向的单元中的内容

3.2 寻址方式

一、概述

◆**寻址方式**：找到存放操作数的地址，把操作数提取出来的方法 。

◆它是计算机的重要性能指标之一，也是汇编语言程序设计中最基本的内容之一。

二、寻址方式说明（7种）

- 立即寻址
- 寄存器寻址
- 直接寻址
- 寄存器间接寻址
- 基寄存器加变址寄存器间接寻址
- 相对寻址
- 位寻址

1、立即寻址

- 指令中给出实际操作数据（立即数）
- 立即数只能作为源操作数
- 与操作码一起存放在程序存储器中
- 一般用于为寄存器或存储器赋常数初值。

8位立即数： MOV A，**#40H** ；A ←40H

16位立即数： MOV DPTR，**#2100H**； DPTR←2100H

2、寄存器寻址

- 指令给出寄存器名，数据在寄存器中
- 指令中源操作数和目的操作数都可以是寄存器寻址

MOV A，R_n ；A←R_n，R_n=R_0 ~ R_7

例如：MOV A，R_0

0100 0000	←	0100 0000
A		R_0

3、直接寻址

- 指令给出存储器单元地址，数据在存储器单元中

例如：MOV　A，40H；　A←(40H), 源操作数是直接寻址

- 直接寻址方式访问SFR、片内数据存储器的低128字节、位寻址空间

思考题：直接寻址方式指令和立即寻址方式指令的形式有什么不同？

4、寄存器间接寻址

- 指令给出寄存器名，寄存器中为地址，地址中的数据为操作数
- 存放地址的寄存器称为间址寄存器或数据指针
- R_i（i=0,1）用作地址指针时，可寻址片内数据存储区的256个单元，但不能访问SFR，也可以访问片外数据存储器的低256个地址单元
- DPTR用作地址指针时，访问片外数据存储器的64KB范围

寄存器间接寻址方式举例：

例：　　MOV A，@R0；A←((R0))

设指令执行前 A=20H，R0=40H，地址为40H存储器单元内容如图所示。执行指令后，

A=　？　,R0 =　？　，(40H)=　？

	41H	67H
R0→	40H	34H

34H　　40H　　34H

5、基寄存器加变址寄存器间接寻址　变址寻址

指令给出的是基址寄存器（ **PC或DPTR** ）和变址寄存器寄存器（累加器A）：

数据地址 = 基地址 + 偏移量

变址寻址思考题：

MOVC　A，@A+DPTR；　　**A ←((A)+(DPTR))**

执行前：A=09H，DPTR=2000H

执行指令后：A=？ DPTR=？

2008H	89H
2009H	12H

变址寻址思考题答案：

MOVC　A，@A+DPTR；　　A ←((A)+(DPTR))

DPTR（执行前）2000H

A（执行前）09H

+

2008H	89H
2009H	12H

A（执行后）12H

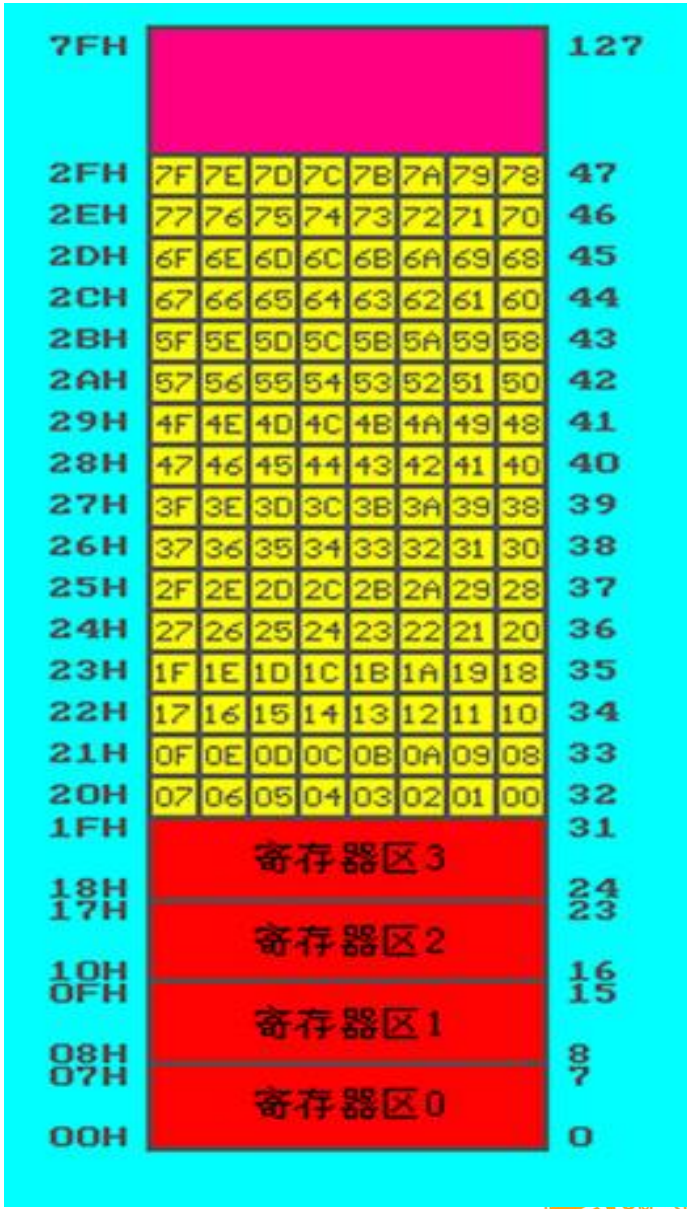
DPTR（执行后）2000H

6、位寻址

- 指令给出位地址，位数据在位地址中
- 片内RAM中的128位和SFR中的93位进行寻址
- MOV C，40H；Cy ←(位地址40H)

指令执行前：Cy=1，位地址40H存储器单元如图

执行指令后：Cy=?



7、相对寻址

- 以程序计数器PC的值加上指令中给出的偏移量“rel”，构成实际操作数的地址

目的地址=当前PC值+rel=指令存储地址+指令字节数+rel

- rel是带符号的单字节数（以补码表示），范围为-128 ~ +127，负数是向前转移，正数是向后转移
- 用于相对转移指令

三、寻址方式说明

- 对程序存储器只能采用基寄存器加变址寄存器间接寻址方式
- 对特殊功能寄存器空间只能采用直接寻址（可以用符号来代表地址），不能采用寄存器间接寻址方式
- （对52单片机）内部数据存储器高128字节，只能采用寄存器间接寻址方式，不能采用直接寻址方式
- 内部数据存储器低128字节既能采用寄存器间接寻址方式，又可以采用直接寻址方式
- 外部扩展的数据存储器只能采用MOVX指令来访问

3.3 51指令系统

- 111条指令，42种操作助记符，描述33种操作功能
- 从功能：数据传送（29）、算术运算（24）、逻辑操作（24）、控制转移（17）、位操作（17）
- 从空间：单字节（49）、双字节（45条）、三字节（17条）
- 从时间：单机器周期（64条）、双机器周期（45条）、四机器周期（2）
- 指令执行时，对程序状态字PSW的标志位有影响

一、数据传送类

- 分为3种：数据传送、数据交换和堆栈操作
- 助记符：MOV、MOVX、MOVC

  XCH、XCHD、SWAP

  PUSH、POP

一、数据传送类（续）　表3-1

(1)MOV指令

MOV 目的, 源　　;目的←源

目的操作数可以是寄存器操作数、存储器操作数、但不能为立即数

源操作数可以是寄存器操作数、存储器操作数、立即数

① 8位二进制数传送

MOV A, R7　　**MOV R2, 70H**

MOV 50H, @R0　　**MOV @R1, 60H**

② 16位二进制数传送

MOV DPTR, #1234H ;DPTR=? DPH=? DPL=?

(2)MOVC指令(变址寻址)，亦称为查表指令

MOVC A, @A+PC ；表格长度不超过256

MOVC A, @A+DPTR ；表格长度不受限制，可在64KB存储器中任意安排，实际编程中更常用

例如以下程序段：

```
MOV  DPTR, #8000H
MOV A, #30H
MOV A, @A+DPTR
```

(3)MOVX指令：实现累加器A与外扩存储器或IO口之间的数据传送，寄存器间址寻址

MOVX A，@Ri

MOVX A，@DPTR

MOVX @Ri，A

MOVX @DPTR，A

外扩存储器地址不超过256时，可以使用Ri；超过256时使用DPTR

```
MOV  DPTR, #8000H
MOVX A, @DPTR
```

```
MOV  DPTR, #8000H
MOV  A, #10H
MOVX  @DPTR , A
```

(4)PUSH、POP堆栈操作指令

堆栈操作只能以直接寻址方式来取得操作数

入栈指令：PUSH direct ；SP ←(SP)+1，(SP) ← (direct)

出栈指令：POP direct ；(direct) ←((SP))，SP ← (SP)-1

PUSH和POP指令应该成对出现

堆栈指令练习

执行下列指令后，SP=？，A=?，B=?

```
ORG 0000H
JMP START
ORG    2000H
START: MOV    A,#02H
       MOV    B,#56H
       MOV    SP,#30H
       PUSH   ACC
       PUSH   B
       MOV    A,#0H
       MOV    B,#01H
       POP    ACC
       POP    B
END
```

SP=30H，A=56H，B=02H

(5) XCH字节交换指令

XCH A, Rn

XCH A, direct

XCH A, @Ri

例如：XCH A, R1

(6) XCHD、SWAP 半字节交换指令

XCHD A, @Ri ；A和源操作数的低4位互换

SWAP A ； A的低4位和高4位互换

例如：XCHD A, @R1

数传指令应用举例

例3-1：片内 RAM 6AH的内容送到片外RAM1000H

```
MOV DPTR, #1000H
MOV A, 6AH
MOVX @DPTR, A
```

例3-2：片外RAM 2000H的内容送到片外RAM1000H

```
MOV DPTR, #2000H
MOVX A, @DPTR
MOV DPTR, #1000H
MOVX @DPTR, A
```

数传指令应用举例

例3-3：片内 RAM 6AH的内容压入堆栈后弹出到50H,设堆栈指针为30H

MOV SP, #30H

PUSH 6AH

POP 50H

例3-4：说明每条指令作用,已知A中内容为34H

MOV R6, #29H

XCH A, R6

SWAP A

XCH A, R6

二、算术运算类指令

- 可处理4种类型的数据：无符号二进制数、带符号二进制数（使用补码形式表示）、无符号压缩十进制数、无符号非压缩十进制数。

二进制数(B)	十六进制数(H)	无符号二进制数(D)	带符号二进制数(D)	无符号非压缩十进制数	无符号压缩十进制数
0000 0111	07	7	+7	7	07
1000 1001	89	137	-119	无效	89
11000101	C5	197	-59	无效	无效

- 加、减、乘、除基本四则运算
- 增量（加1）、减量（减1）运算
- 重点助记符：ADD、ADDC、INC、DA

  SUBB、DEC、

  MUL、DIV

1、加法、减法指令

（1）不带进位加法

ADD A，源操作数；A←（A）+源，影响C, OV、AC、P

例如：（A）=53H, (70H)=0FCH

ADD A, 70H ；(A)=4FH, C=1, AC=0, OV=0, P=1

例如：（A）=85H, (R1)=30H, (30H)=0AFH

ADD A, @R1 ；(A)=34H, C=1, AC=1, OV=1, P=1

思考：ADD A, R1执行结果?

（2）带进位加法： ADDC A，源操作数; A←(A)+源+C，影响C、OV、AC、P

带进位加法指令ADDC用于多字节运算

例如： （A）=85H, (20H)=0FFH,C=1

ADDC A, 20H ；(A)=85H, C=1, AC=1, OV=0, P=1

(3)带借位减法指令

SUBB A，源操作数；A←(A)-源-C，

；影响C、OV、AC、P

例如：（A）=85H, (20H)=34H,C=1

SUBB A, 20H ；(A)=50H, C=0, AC=0, OV=1, P=0

(4)、增量(加1)指令

INC 目的操作数 ； 不改变PSW状态

例如：（A）=0FFH,（R3）=0FH, (30H)=0F0H,（R0）=40H, (40H)=00H,

INC A ; (A)=00H

INC R3 ; (R3)=10H

INC 30H ;(30H)=0F1H

INC @R0 ;(R0)不变；（40H）=01H

注意：当用本指令修改P0~P3, 其功能是读-修改-写指令。

(5)、减量(减1)指令

DEC 目的操作数 ；不影响PSW

例如： （A）=0FH,（R3）=19H, (30H)=00H, （R0）=40H, (40H)=0FFH,

DEC A ; (A)=0EH

DEC R3 ; (R3)=18H

DEC 30H ;(30H)=0FFH

DEC @R0 ;(R0)不变； （40H）=0FEH

类似：当用本指令修改P0~P3, 其功能是读-修改-写指令。

（6）十进制调整指令：对累加器参与的BCD码加法的和进行十进制调整，P42图3-3

DA　A；对A进行十进制调整

例如：（A）=56H,（R5）=67H

ADD A, R5

DA A；

执行结果：(A)=23H, C=1

(7) 乘法指令

MUL AB ；**BA←(A)×(B)，C←0**

；乘积超过255（0FFH）时， OV＝1

；C总是清0

例如：（A）=50H,（B）=0A0H

MUL AB

执行结果：(A)=32H, (A)=00H(即积=3200H), C=0,OV=1

（8）除法指令

DIV AB ；A÷B，A←商，B←余数，C←0

；当除数B=0，OV←1；B≠0，则OV←0

例如：（A）=0FBH,（B）=12H

DIV AB

执行结果：(A)=0DH, (A)=11H, C=0,OV=0

指令应用举例：例3.5—例3.13

三、逻辑运算类指令

- 包括与、或、异或及清除、求反、移位等操作
- 助记符：ANL、ORL、XRL、CLR、CPL、RL、RLC、RR、RRC、

1、双操作数指令

与指令：ANL，常用场景：使目标操作数某些位清零（与‘0’与），其他位保持不变（与‘1’与）

或指令：ORL，常用场景：使目标操作数的某些位不变（与‘0’或），其它位置为“1”（与‘1’或）

异或指令：XRL，常用场景：使目标操作数的若干位不变（与‘0’异或），其它位“求反”（与‘1’异或）的场合。

例如：若A=01×××××××B，×表示随机状态，为1或0，执行下述一组指令执行后A的值如何?

XRL A，#0C0H；累加器A的D_7、D_6取反

ORL A，#03H ；累加器A的D_1、D_0置1

ANL A，#0E7H；累加器A的D_4、D_3清0

A=10×00×11B

思考：如何将累加器A中的数据高4位清0，低位不变？
如何将寄存器R2中的数据奇数位取反，偶数位不变？

- 逻辑运算类指令举例：P46 例3-12~3-15

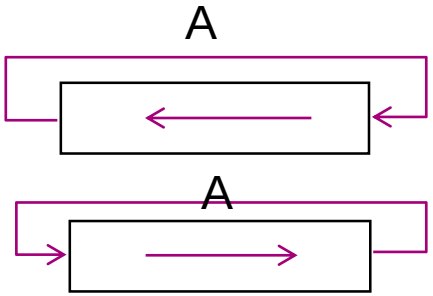
2、单操作数指令

A清零：CLR A ；A←0，只影响P标志位，其他不影响

A取反：CPL A ；A←/A，不影响标志位

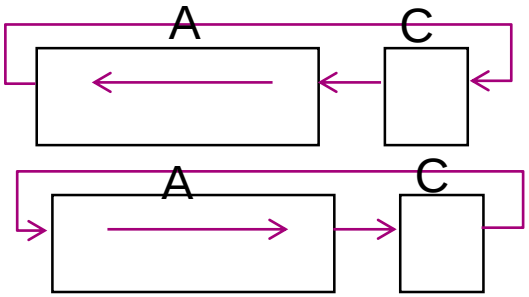
RL A；A循环左移1位，不影响标志位

RR A；A循环右移1位，不影响标志位



RLC A；带C循环左移1位，不影响AC、OV

RRC A；带C循环右移1位，不影响AC、OV



单操作数指令举例

设(A)=0F3H, C=1, 顺序执行以下指令后A和C分别为：

CPL　A; (A)=0CH, C=1

RL　A; (A)=18H, C=1

RLC　A; (A)=31H, C=0

RR　A; (A)=98H, C=0

RRC　A; (A)=4CH, C=0

CLR　A; (A)=00H, C=0

四、控制转移类指令

- 改变程序运行的方向
- 包括无条件转移、条件转移、调用和返回指令等。
- 助记符：AJMP、LJMP、SJMP、JMP、JZ、JNZ、CJNZ、DJNZ、ACALL、LCALL、RET、RETI、NOP

1、无条件转移指令

AJMP addr11 绝对短跳转指令。指令转移范围：**2KB，为了与MCS-48兼容，51中基本不用**

LJMP addr16 长跳转指令，跳转到addr16指向的地址。指令转移范围：**64KB**

SJMP rel 相对短跳转指令。指令转移范围：**-128-+127**

例如：THISL: SJMP THATL; 跳转到THATL

设THISL处地址为0100H，THATL处地址为0155H，则：0100+2+rel=0155H，得到rel=53H

1、无条件转移指令（续）

JMP @A+DPTR 间接跳转指令。 **跳转到A+DPTR之和指向的地址**

指令转移范围：**64KB，该指令不改变A和DPTR的内容，也不影响标志位，相加结果送PC**

2、条件转移指令

判零跳转指令：

JZ rel ；累加器A为0跳转；否则继续

JNZ rel ；累加器A不为0跳转；否则继续

例如：设（A）=02H，且程序中未修改A的内容，则：

```
MAIN:  …
       …
       JNZ     NEXT
       …
NEXT:  …
```

程序执行到条件转移指令时，会跳转到NEXT处

比较不相等转移指令：

CJNE A，n，e ；若A≠(n)，则跳转；否则继续

例如：CJNE A, #60H, NEQ; (A) ≠ 60H转移到NEQ

```
EQ:   …        ;   (A) 等于60H处理程序
      …
NEQ:  JC LOW       ; (A)小于60H转移
      …            ; (A)大于60H处理程序
LOW:  …            ; (A)小于60H处理程序
```

减1不为0跳转指令（循环转移指令）DJNZ

（源操作数）=（源操作数）-1, 若结果不为0跳转，否则继续

例：用于循环结构程序。设要求程序循环执行100次

```
        MOV R2，#100 ；设循环计数器初值
LOOP:   …            ；多次循环程序段
        DJNZ R2，LOOP ；循环控制
        …            ；循环结束
```

思考：当循环计数器初值为0，循环次数有多少？

DJNZ用于延时子程序为：（P1.1输出方波）

```
START: SETB P1.1; P1.1置位
DL: MOV    30H,    #03H;
DL0:       MOV    31H, #0F0H;
DL1:       DJNZ 31H, DL1 ; (31H)=(31H)-1,不为0重复
           DJNZ  30H, DL0 ;  (30H)=(30H)-1,不为0跳转到DL0
            CPL P1.1;  P1.1取反
           SJMP DL
```

3、子程序调用及返回指令

- 子程序调用和返回指令也使程序发生转移
- 子程序调用过程与转移指令不同：调用子程序之前，先用堆栈保存断点地址（保护现场）；返回时取出断点地址（恢复现场）

3、子程序调用及返回指令)

长调用指令： LCALL addr16 指令调用范围：64KB

(PC)←(PC)+3，

断点入栈：(SP)←(SP)+1, ((SP))←$(PC)_L$ ；

(SP)←(SP)+1, ((SP))←$(PC)_H$

(PC) ←addr16

绝对调用指令： ACALL 指令调用范围 2KB，不常用

例如：（SP）=60H，标号START为0100H，标号MIR为8100H，执行指令：

START: LCALL MIR;

结果： (SP)=62H, (61H)=03H, (62H)=01H, (PC)=8100H,

子程序返回指令

RET 子程序返回

$(PC)_H \leftarrow ((SP))$，$SP \leftarrow SP-1$,

$(PC)_L \leftarrow ((SP))$，$SP \leftarrow SP-1$,

指令从堆栈弹出保存的PC地址，实现子程序返回

RETI 中断返回

$(PC)_H \leftarrow ((SP))$，$SP \leftarrow SP-1$,

$(PC)_L \leftarrow ((SP))$，$SP \leftarrow SP-1$,

指令从堆栈弹出保存的PC地址，实现中断返回

例如：设(SP) =62H, (62H)=07H,(61H)=30H,执行指令

RET

结果： (SP) =60H, (PC)=0730H，程序回到0730H继续执行

子程序调用及返回指令举例

程序执行后，

SP= 42H

A= 30H

B= 30H

```
                ORG     2000H
2000H:          MOV     SP,     #40H
2003H:          MOV     A,      #30H
2005H:          LCALL           2500H
2008H:          ADD     A,      #10H
200AH:          MOV     B,      A
200CH: L1:      SJMP    L1
                ORG     2500H
2500H:          MOV     DPTR,   #200AH
2503H:          PUSH    DPL
2505H:          PUSH    DPH
2507H:          RET
```

空操作指令 NOP

- 例3-16，3-17

五、位操作类指令

1、位传送指令

MOV C，b ； C←(b)

MOV b，C ； (b)←C

例如：MOV C, 20H； C ← (位地址20H)

2、位变量修改指令：清零、置1、取反

CLR、SETB、CPL

CLR C ； C ← 0，

CLR 40H ； (位地址40H) ← 0

3、位逻辑运算指令：

ANL、ORL

ANL C，40H ；C←C∧(40H)

ANL C，/40H ；C←C∧/(40H)

OR C，P1.2 ；C←C∧P1.2

4、位变量条件转移指令

判C转移指令：

JC ： C=1转移，否则继续

JNC ： C=0转移，否则继续

判位转移指令：

JB ： 直接寻址位为1转移，否则继续

JNB ： 直接寻址位为0转移，否则继续

判位清零转移指令：

JBC ： 如果直接寻址位为1转移，并对该位清0；否则继续

位操作指令举例：例3-18，3-19

3.4 伪指令

汇编语言源程序包含一些不能被单片机CPU执行的指令，称为“汇编伪指令”。伪指令是对汇编起某种控制作用的特殊命令，其格式与通常的操作指令一样，并可加在汇编程序的任何地方，但它们并不产生目标代码。

汇编：汇编语言源程序翻译为机器语言程序（目标代码）的过程

- **ORG： 定位伪指令　　org 0100h**

—— 定义程序、数据汇编存放时的起始地址。

- **END：汇编语言程序结束伪指令**

—— 一般用在程序的末尾，表示程序结束。

- **DB：定义字节伪指令**

格式：标号: DB 字节常数/字符/表达式

例如：data1：db 2，3，4，5*6

- **DW：定义字伪指令**

格式：标号: DW 字或字串，

大端存储模式（高8位在前，低8位在后）

例如：data2：dw 2，3，4，5

- **DS：定义存储器空间伪指令**

例如：data1： ds 07h; 保留7B空间

- **EQU：赋值伪指令**

例如：PI　　EQU　3

- **BIT：定义位地址符号伪指令，**

格式：字符名称　　BIT　位地址

例如 P11　　BIT　　P1.1

本章作业：

1、指令应用举例：例3.1—例3.7，例3.9，例3.12—例3.16，例3.18-3.19；

2、课后习题：2，3，4，5，

3、课堂练习：6，7